

HASH TABLES + FIRST RECURSION

"From $O(n^2)$ to $O(n)$ for free — and the first real recursive function."



■ WHERE WE ARE ■

Last class we walked through linked lists, one node at a time. Today we change subject: **hash tables**. Then, at the end, we plant a small seed of recursion. The big recursion lecture comes next.

Today's plan

Hash tables — refresher with two killer applications

~20 min

One small recursive function — **factorial(n)**

~5 min

Time

Why this mix?

Hash tables are a quick wake-up — you've used **dict** a hundred times. We don't need a full hour. So we use the leftover time to introduce recursion gently, right before the deep dive.

"Two subjects today. One you already know. One that's about to change everything."

■ HASH TABLES — $O(1)$ LOOKUP ■

A hash table stores key $\rightarrow$ value pairs and gives you the value for a key in **$O(1)$ on average**. That's the magic. In Python, you already use one every day — it's called **dict**.

```
d = {} # an empty hash table
d["apple"] = 1 # insert  $\rightarrow O(1)$ 
d["banana"] = 2
d["apple"] # lookup  $\rightarrow O(1)$ 
"apple" in d # membership  $\rightarrow O(1)$ 
del d["banana"] # delete  $\rightarrow O(1)$ 
```

Efficiency in Practice

Every line in the example runs in **$O(1)$** . No scanning the entire structure, no walking through nodes.

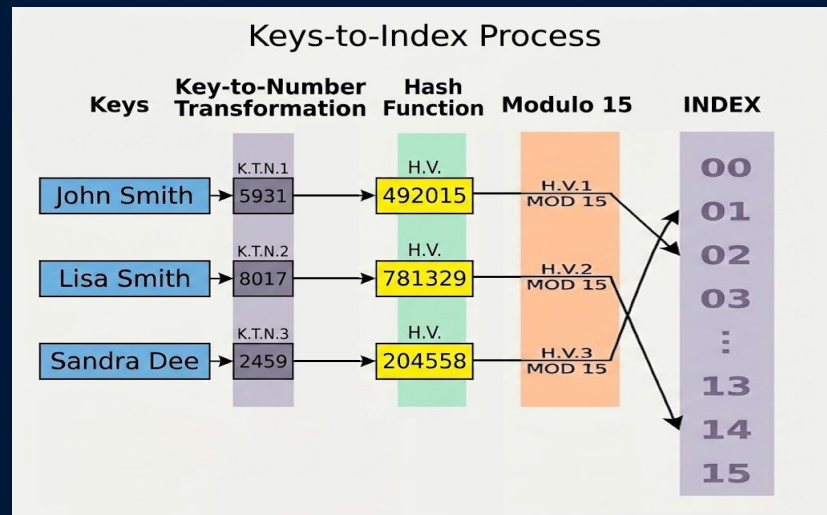
One hash calculation leads to **one direct hit** in memory.

- Instant lookups regardless of size
- Constant time insertions
- Fundamental to high-performance software

"You've been using hash tables all year. Today we look under the hood."

■ HOW IT WORKS ■

Inside, the **dict** is just an array. To store a pair, Python turns the key into a number, then into an index. To find a key later, it redoes the same math and goes straight to that index — no scanning.



[01] Compute hash(key)

A function that turns the key into a number.

[02] Take it modulo array size

That gives an index inside the array.

[03] Store at array[index]

Put the pair there. Direct hit, every time you look it up.

[04] Collisions happen

Two keys can land on the same index. The dict handles it internally — you can trust $O(1)$ on average.

"One hash. One index. One direct hit. That's why dict is fast."

■ LIST vs DICT — THE GAP ■

Same operations, very different speed. The reason most "find" problems can be sped up: **a dict turns scanning into a direct hit.**

Operation	List	Dict
`x in container`	$O(n)$	$O(1)$
Get value	—	$O(1)$
Insert	$O(1)$ at end	$O(1)$

If you ever write:

```
for x in lst:  
    if x == target: ...
```

Ask yourself: **should I use a dict instead?**

A dict turns scanning into a direct hit.

"Scanning is $O(n)$. Hashing is $O(1)$. Same problem, very different bill."

■ COUNTING — THE D.GET PATTERN ■

The most common use of a **dict** in your daily code: counting things. How many times each word appears or user clicked. The pattern fits in **three lines**.

```
def word_count(text):  
    d = {}  
    for w in text.split():  
        d[w] = d.get(w, 0) + 1 # the magic line  
    return d  
  
# {"the": 3, "quick": 2, "brown": 2, "fox": 2, ...}
```

[01] d.get(w, 0)

Returns `d[w]` if `w` is already a key. Otherwise returns `0`.

[02] + 1

We just saw `w` one more time.

[03] d[w] = ...

Store the new count. One line, three things at once.

"d[k] = d.get(k, 0) + 1 — memorise it. You'll write it 100 times this year."

■ TWO-SUM — THE NAIVE WAY ■

Given a list of numbers and a target, find two indices $i < j$ such that `nums[i] + nums[j] == target`. The first reflex: try every pair. Two nested loops.

```
def two_sum_naive(nums, target):  
    for i in range(len(nums)):  
        for j in range(i + 1, len(nums)):  
            if nums[i] + nums[j] == target:  
                return (i, j)  
    return None  
  
# nums = [2, 7, 11, 15], target = 9  
# → 2 + 7 = 9 → return (0, 1)
```

Works. But on **100,000** numbers → about **5 billion** operations. Way too slow.

Two nested loops over the same list = $O(n^2)$.

"Nested loops over the same list. That's $O(n^2)$. On big inputs, it dies."

■ TWO-SUM — $O(n^2) \rightarrow O(n)$ ■

ONE pass through the list. As we walk, we ask one question per element: *"Have I already seen 'target - x' somewhere before?"*

A **dict** answers in **$O(1)$** . If yes, we found the pair. If no, we remember x for later.

```
def two_sum(nums, target):  
    seen = {} # value → index  
    for i, x in enumerate(nums):  
        complement = target - x  
        if complement in seen: #  $O(1)$  lookup!  
            return (seen[complement], i)  
        seen[x] = i # remember x  
    return None
```

100,000 numbers $\rightarrow$ 100,000 operations. **50,000× faster** than the naive version.

Hash tables turn $O(n^2)$ into **$O(n)$** by replacing "scan the rest" with **"check a dict in $O(1)$ "**.

"About to write a nested loop? Ask: could a dict remember things and collapse this?"

■ RECURSION — A FIRST TASTE ■

Now we switch subject. The deep dive on recursion is next class — today we plant a seed. A **recursive** function is a function that calls itself. To work, it needs two pieces — skip either one and it breaks.



[01] Base case

A condition where the function returns directly, without recursing. **Without it, the function runs forever and Python crashes.**

[02] Recursive case

The function calls itself on a smaller, simpler input.

[03] The reflex

Write the base case FIRST. Then write the recursive call.

"Two pieces, every time: a base case to stop, a recursive case to shrink."

■ FACTORIAL — THE CLASSIC ■

factorial(n) = $n \times (n-1) \times \dots \times 2 \times 1$. By convention,
factorial(0) = 1.

The trick: notice that **factorial(n) = $n \times \text{factorial}(n-1)$** . If you can compute the smaller one, the bigger one is just one multiplication.

```
def factorial(n):  
    if n == 0: # ← base case  
        return 1  
    return n * factorial(n - 1)  
            # ↑ recursive case
```

Two lines do all the work.

01. Base Case: The first line is the base case—stop and return 1.

02. Recursive Case: The second is the recursive case—call yourself on a smaller n .

Recursion breaks a big problem into a smaller version of itself.

"Spot the two pieces? Base case on top. Recursive call below. Every recursion looks like this."

■ TRACE — FACTORIAL(4) ■

Each call **waits** for the next one to finish. Python keeps track of all of them — that's called the **call stack**. We'll visualise it properly next class. For now, just watch the calls roll in, then roll back.

```
factorial(4)
→ 4 * factorial(3)
  → 3 * factorial(2)
    → 2 * factorial(1)
      → 1 * factorial(0)
        → 1 # base case hits!
          ← 1 * 1 = 1
            ← 2 * 1 = 2
              ← 3 * 2 = 6
                ← 4 * 6 = 24
```

"Each call waits. The base case hits. Then the answers roll back up."

■ FORGOT THE BASE CASE? ■

Drop the base case and your function never stops. Python keeps calling, calling, calling — until it hits the recursion limit and **crashes**.

```
def factorial(n):  
    return n * factorial(n - 1) # no base case!  
  
factorial(4)  
# → factorial(3) → factorial(2) → ... →  
factorial(-1) → ...  
# → RecursionError: maximum recursion depth exceeded
```

Reflex to build today:

When you write a recursive function, write the base case **FIRST** — before the recursive call.

Type the **stop condition**, then add the call.

"No base case = no stop = crash. Write the stop condition first, every time."

■ RECAP — AND WHAT'S NEXT ■

Today, in five lines:

- **Hash table** = $O(1)$ average lookup. That's why **dict** is your friend.
- **Counting pattern**: `d[k] = d.get(k, 0) + 1`. You'll write it everywhere.
- **Two-sum**: a dict turns $O(n^2) \rightarrow O(n)$ by replacing scanning with one $O(1)$ check.
- **Reflex**: nested loops? Ask if a dict could collapse them.
- **Recursion** = function that calls itself + a base case. Today was just a taste.

The road ahead:

Now → hash tables + first recursive function

Next (S4) → Recursion deep dive — call stack, patterns, classical bugs

Then (S5+) → Trees, BST, heaps — every meaningful tree op is recursive

"Next class: recursion in depth. We'll visualise the call stack and learn the bugs that catch every first-year. After that, you're ready for trees."

"Type one recursive function today. Recursion will feel familiar next class."